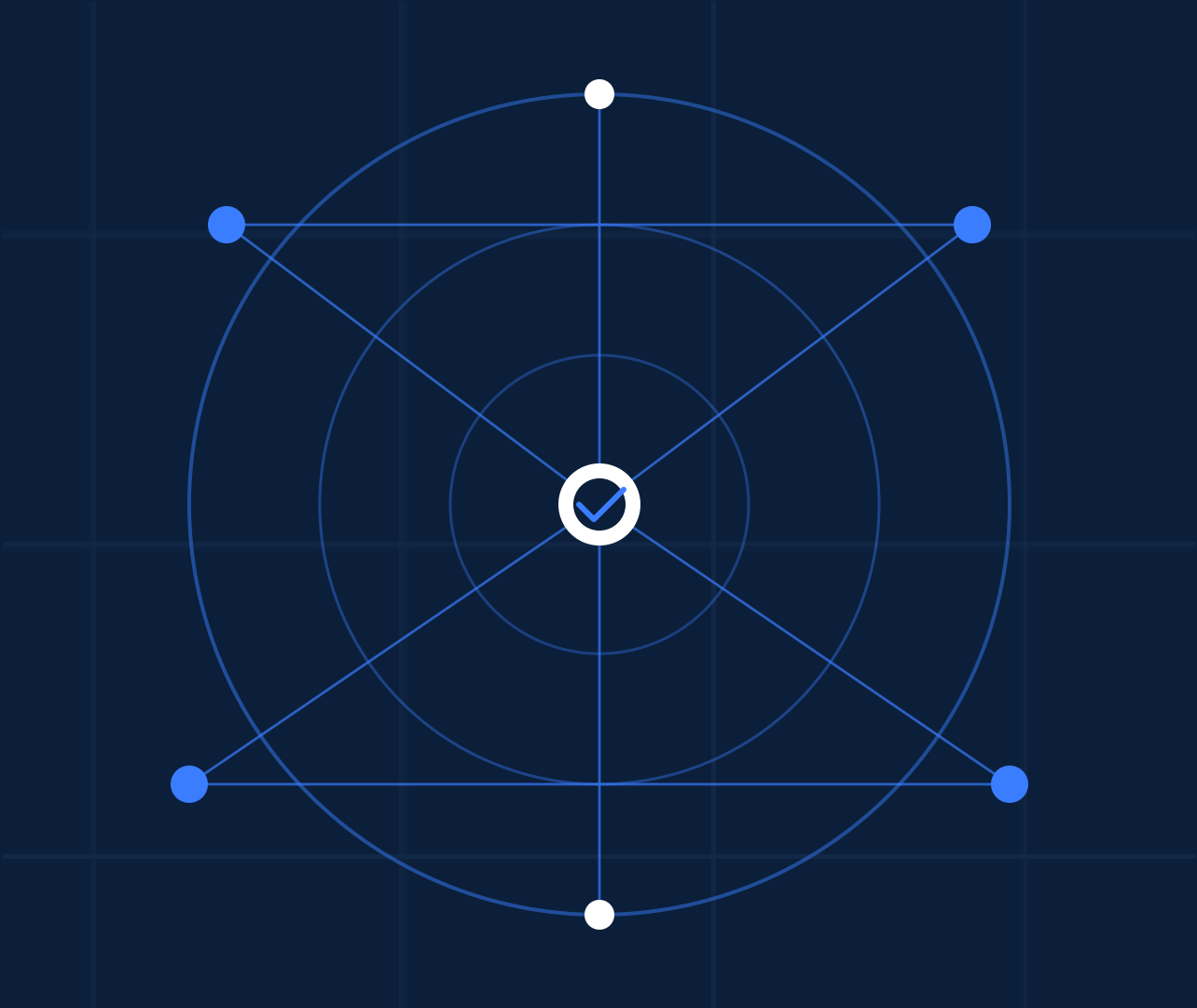




A LAYERED PROOF OF RESERVES FRAMEWORK

LPOR

Usable and Publicly Auditable Solvency Verification

AUTHORS

Donggoo Kim
Georgia Institute of Technology

Rajesh Upadhayaya
University of New Mexico

Milosz Bator
Georgia Institute of Technology

Tao Le
Georgia Institute of Technology

PRESENTED BY

Donggoo Kim · dkim3055@gatech.edu

When trust fails at global scale

Most everyday users still access crypto through custodians — and custodial failures have occurred throughout the industry’s growth.

GROWTH OF THE CUSTODIAL ERA

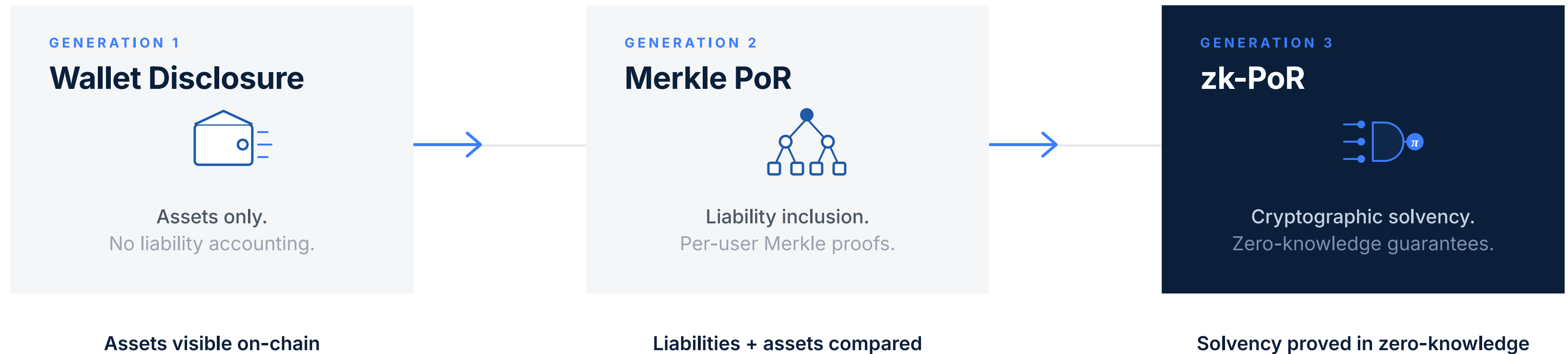


THE MOTIVATING QUESTION

How can users actually know that an exchange holds their funds?

The evolution of Proof of Reserves

Three generations, each cryptographically more sophisticated than the last.



OBSERVATION

Each generation improved cryptographic sophistication.

What does Classic Merkle PoR verify?

Proof of Reserves tries to support one solvency claim: **Total user liabilities** ≤ **CEX on-chain assets**.

STEP 01

Inclusion check

QUESTION

"Is my balance included in the committed liability set?"

> `verify_merkle_path(user_proof.json)`

WHO

User

STEP 02

Root consistency check

QUESTION

"Am I checking against the same committed dataset as everyone else?"

> `verify_merkle_root(snapshot_root.json)`

WHO

User

STEP 03

Solvency check

QUESTION

"Are total user liabilities covered by exchange reserves?"



Auditor verifies privately

WHO

Auditor (with private record access)

TWO BOTTLENECKS

Users must **participate** to detect omission. Auditors are needed for **aggregate solvency**.

Where each PoR design pays its cost

Merkle pays in trust, zk pays in opacity, LPOR pays in size.

	GENERATION 2 Classic Merkle PoR	GENERATION 3 zk-PoR	THIS WORK LPOR
OMISSION DETECTION	User participation	User participation	Easy user participation
AGGREGATE SOLVENCY	Internal auditor	Cryptographic circuit	Public sum
MAIN LIMITATION	Needs internal auditor User runs script	Technically complex User runs script	Larger proof size Partial privacy trade-off
USER'S COMMITMENT VIEW	0x7a81... 0x4c92... 0xa117...	C = 0x92ab... π = 0x7f19...	0x7a81... • TBTC-1 0x4c92... • TBTC-0.1 0xa117... • TBTC-0.01

Three system properties — three different things users see.

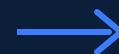
Usability is a **security parameter**.

A proof's effective strength depends on how many independent users can — and do — verify it.

STEP 01

Better usability

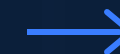
Verification a non-technical user can complete in seconds.



STEP 02

More participation

Independent verifiers grow from a handful to thousands.



OUTCOME

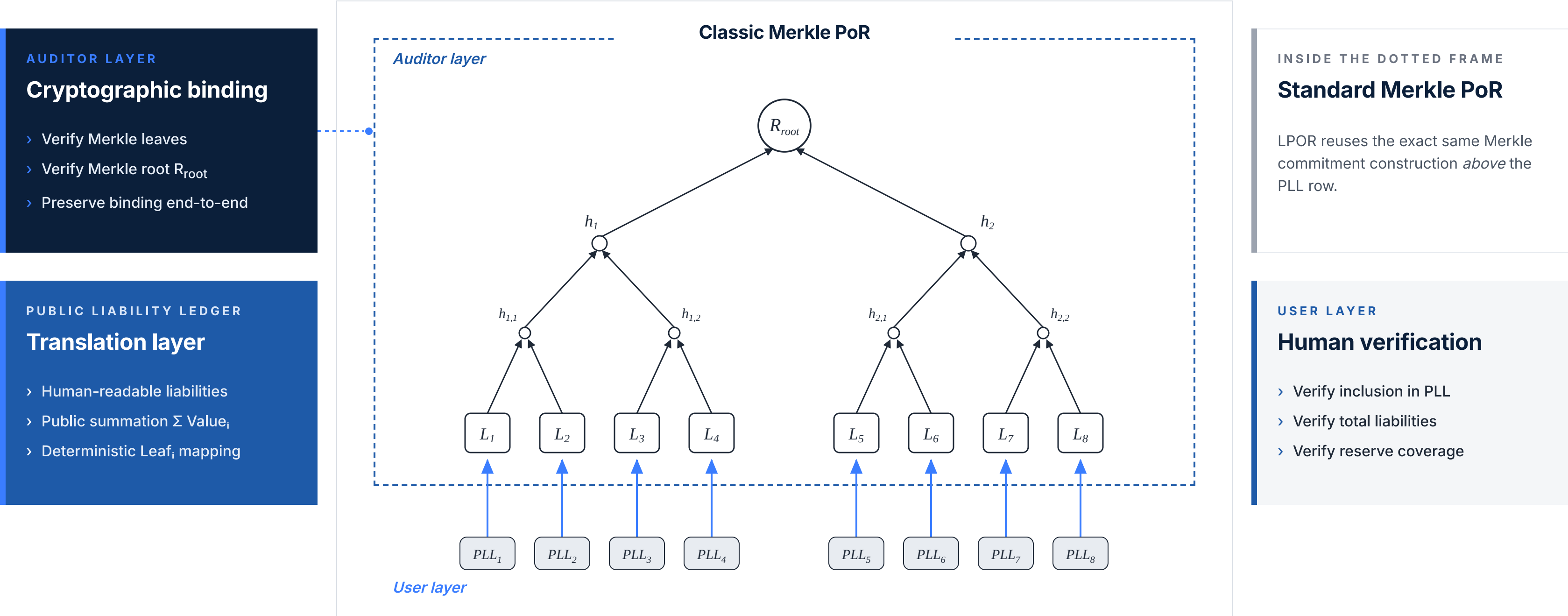
Higher fraud detection

Omissions exposed with high probability across users.

$$P_{\text{detect}} = 1 - (1 - p)^k$$

LPOR architecture: Public Liability Ledger + Layered Verification

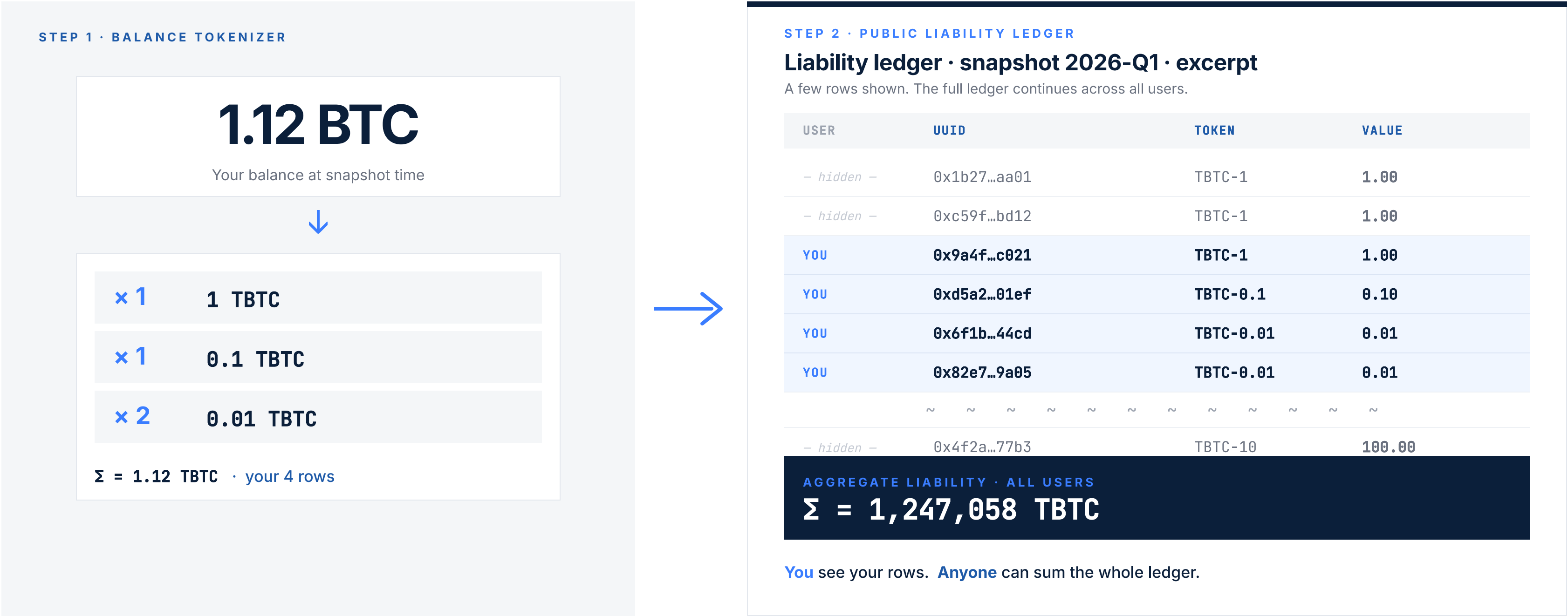
Users see liabilities at the bottom. Auditors verify commitments at the top. Both anchored to one Merkle root.



Users verify liabilities at the PLL row. Auditors verify commitments up the tree. Both meet at the same Merkle root.

From balance to Public Liability Ledger

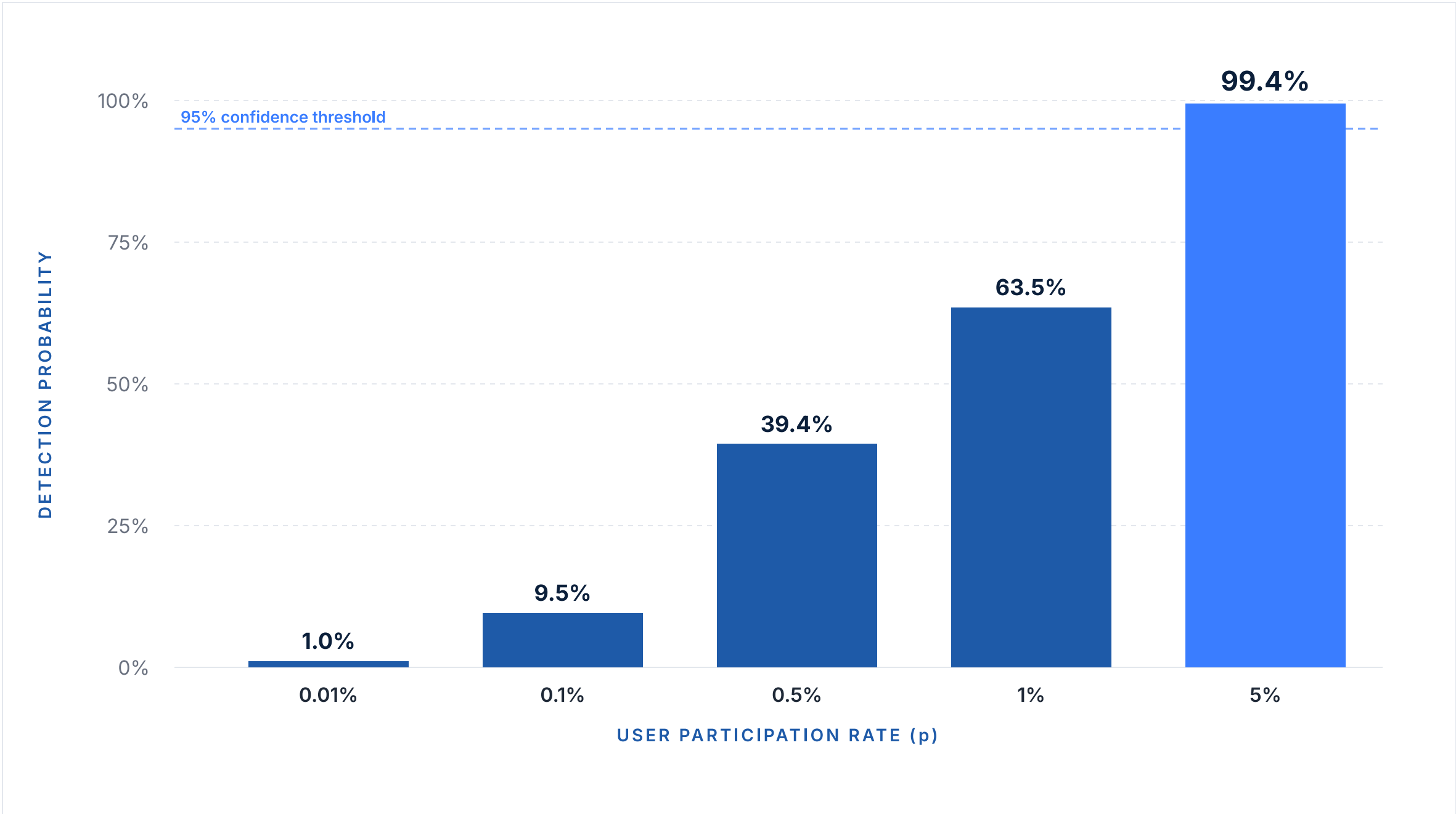
Your balance becomes a few rows in a global, public, summable ledger.



Better usability leads to better security

The cryptography does not change. Detection probability improves because participation increases.

Simulated: 10M users × 0.001% omission = **k = 100 omitted users**.



HEADLINE RESULT

99.4%

Omission detection probability

requires only
5%

of users to actually verify

MODEL

$$P_d = 1 - (1 - p)^k$$

Key takeaways

01

Classic Merkle PoR still relies on auditor-side aggregate liability verification.

02

zk-PoR removes auditor trust through cryptographic circuits.

03

LPOR removes auditor trust through public liability recomputation.

04

In PoR, usability can affect practical security through participation.

Cryptographic soundness is important.

But a proof that nobody verifies is not **transparency**.

Thank you · Questions?

Donggoo Kim · dkim3055@gatech.edu

IEEE ICBC 2026